InterviewIQ — Problem Statement & Project Milestones

1. Problem Statement

Hiring teams often want a fast, first-pass assessment of a candidate's interview answers before a human reviewer evaluates them.

The goal of this project is to build **InterviewIQ**, an AI-powered mock-interview coach with a working browser interface.

Given a candidate's spoken or written answer to an interview question, InterviewIQ should:

- Evaluate the answer using a set of tools.
- Produce short, encouraging feedback.
- Remember the candidate's evaluations throughout the entire interview session.
- Answer questions about the candidate's progress at any point during the session.
- Provide meaningful aggregated insights rather than simply recapping previous answers.
- Surface all of this functionality through a browser-based UI rather than only through a terminal.

---

2. Project Requirements

Ask 1 — Build a Tool-Calling Evaluator Agent

Implement an evaluator agent that can decide which evaluation tools are relevant to a candidate's answer and call them as needed.

The project requires three rule-based tools. Each tool must return a structured dictionary.

1. `detect_filler_words`

Detect and flag common filler words such as:

- "um"
- "like"

- "basically"

The tool should identify filler words present in the candidate's answer and return the results in a structured format.

2. `check_star_structure`

Evaluate whether a behavioral interview answer covers the four components of the STAR framework:

- **Situation**
- **Task**
- **Action**
- **Result**

The tool should determine which components are present or missing.

3. `score_relevance`

Score the relevance of the candidate's answer on a scale of **0–100**.

The score should be based on how many expected keywords or concepts related to the interview question appear in the candidate's answer.

Evaluator Agent Behavior

The evaluator agent should:

1. Read the candidate's answer.
2. Determine which evaluation tool(s) are relevant.
3. Call the appropriate tool(s).
4. Interpret the tool results.
5. Generate short, encouraging feedback for the candidate.

---

3. Ask 2 — Session Memory and Aggregated Evaluation

The agent must maintain memory across the **entire interview session**.

It should remember every answer and its corresponding evaluation, rather than considering only the current answer.

The agent must also be able to answer meta-questions at **any point during the session**.

Examples include:

- "How am I doing so far?"
- "What's my weakest area?"
- "What should I improve?"
- "How has my performance been across the questions?"

The agent must not wait until the end of the interview to provide these insights.

Required Aggregation

The final and intermediate reports must perform actual aggregation of the session data.

At minimum, the report must include:

Average Relevance Score

Calculate the average relevance score across all questions answered so far.

Weakest Area

Identify the **question with the lowest relevance score** and name it as the candidate's weakest area.

This must be based on the complete session history, not simply the most recently evaluated answer.

Memory Verification

The project includes a verification script:

`memory_check.py`

Run the script against the completed agent.

The script:

1. Evaluates one strong answer.

2. Evaluates one deliberately weak answer.
3. Asks the agent a meta-question about performance.
4. Checks whether the agent correctly identifies the weak answer's question as the weakest area.

Success Criteria

The memory check passes if the agent correctly identifies the deliberately weak answer's question as the **weakest area**.

If the agent instead identifies the most recently asked question regardless of its score, the implementation is only looking at the last turn and does not have working session-level memory and aggregation.

---

4. Ask 3 — Working Gradio Interface

Build a browser-based interface using **Gradio**.

The UI is a required part of the project and must demonstrate the complete InterviewIQ workflow.

The interface should include the following components:

Current Question

Display the interview question currently being answered.

Answer Box

Provide a text input area where the candidate can enter their answer.

Feedback Panel

Display the evaluator's feedback after an answer is submitted.

Live Scorecard

Show every question answered so far along with its evaluation results.

The scorecard should update throughout the session.

Coach / Meta-Question Box

Provide a free-form input where the candidate can ask questions such as:

- "How am I doing so far?"
- "What is my weakest area?"
- "What should I focus on improving?"

The agent should answer these questions using the **entire session history**.

Final Report Button

Provide a button that generates the final aggregated interview report.

The report should include meaningful session-level insights, including:

- Average relevance score.
- Weakest area.
- Overall performance observations.

---

5. Ask 4 — Bonus: Multi-Agent System

This requirement is optional and is not graded.

None of the six classes actually built a complete multi-agent system.

Class 6's guide introduces the terminology of an **orchestrator** coordinating **specialized agents**, but explicitly scopes out building a complete multi-agent system as a larger topic for later.

Therefore, this requirement is intentionally outside the core project.

The project includes the following fully worked files:

- `interviewer_agent.py`
- `run_multi_agent.py`

These files demonstrate a second agent called the **Interviewer**.

Interviewer Agent

The Interviewer agent's only responsibility is to:

1. Look at how the candidate has performed so far.
2. Decide which category should be tested next.

3. Select the next type of interview question.

The selected question is then handed off to the existing **Evaluator agent** from Asks 1–3.

The Evaluator agent itself remains unchanged.

Orchestrator Loop

A small orchestrator coordinates the interaction between the two agents:

Interviewer Agent → Selects Next Category → Question → Candidate Answer → Evaluator Agent → Evaluation → Interviewer Agent → Next Category

You can run the multi-agent example using:

```
python run_multi_agent.py
```

You are encouraged to read, run, and extend this implementation if you want to explore multi-agent orchestration further.

However, there is **nothing required to build or modify for this bonus section**.

---

6. Suggested Implementation Approach

Follow the milestones in this order to reduce debugging complexity.

Step 1 — Get the Plumbing Working

Before implementing the evaluation tools:

- Set up the environment.
- Configure the API key.
- Make one plain LLM call.
- Confirm that the API/provider configuration works correctly.

Step 2 — Implement the Tools Independently

Write and test each tool as a standalone Python function.

Do not immediately connect the tools to the agent.

First make sure that:

- `detect_filler_words` works correctly.
- `check_star_structure` works correctly.
- `score_relevance` works correctly.

Step 3 — Connect the Tools to the Agent

Wire all three tools into the evaluator's tool-calling loop.

Verify that one complete evaluation works end-to-end using:

`main.py`

Step 4 — Add Session Memory

Once the evaluator works correctly:

- Store every evaluated answer.
- Store the corresponding scores and evaluation results.
- Ensure previous answers remain available during later turns.

Test the agent with several interview questions and verify that the session history accumulates correctly.

Step 5 — Implement Aggregated Reporting

Implement:

`generate_final_report`

The report must perform real aggregation, including:

- Average relevance score.
- Lowest-scoring question.
- Named weakest area.

Step 6 — Implement Mid-Session Questions

Implement:

`ask_agent`

This allows the candidate to ask meta-questions while the interview is still in progress.

The answers must be based on the **complete session history**, not only the most recent interaction.

Step 7 — Run the Memory Check

Run:

```
python memory_check.py
```

Do not move on until the memory check correctly identifies the deliberately weak answer as the weakest area.

Step 8 — Build the Gradio UI

Only after the underlying agent is working reliably should you build:

```
app.py
```

The UI should simply provide a browser-based layer on top of the tested agent.

---

7. Project Milestones

The project can be completed through the following milestones:

Milestone 1 — Environment Setup

- Create a Python virtual environment.
- Install dependencies using `pip install -r requirements.txt`.
- Configure `.env` with the required API key.

Milestone 2 — Plain LLM Call

Make a basic LLM call without tools.

This confirms that the API key, provider, and environment are configured correctly.

Milestone 3 — Filler Word Detection

Implement and directly test:

```
detect_filler_words
```

Milestone 4 — STAR Structure Detection

Implement and directly test:

`check_star_structure`

Milestone 5 — Relevance Scoring

Implement and directly test:

`score_relevance`

Milestone 6 — Tool-Calling Evaluator

Connect all three tools to the evaluator's tool-calling loop.

Verify one complete evaluation end-to-end using:

`main.py`

Milestone 7 — Session Memory

Add persistent session memory.

Verify that multiple questions and evaluations accumulate correctly.

Milestone 8 — Final Report

Implement:

`generate_final_report`

The report must calculate:

- Average relevance score.
- Weakest area based on the lowest-scoring question.

Milestone 9 — Mid-Session Agent Questions

Implement:

`ask_agent`

The agent must answer meta-questions using the complete session history.

Milestone 10 — Memory Verification

Run:

```
python memory_check.py
```

The test must correctly identify the deliberately weak answer as the weakest area.

Milestone 11 — Gradio Application

Build:

```
app.py
```

Verify the complete browser-based workflow:

Question → Answer → Evaluation → Feedback → Scorecard → Meta-Question → Final Report

Milestone 12 — Bonus Multi-Agent System

Run:

```
python run_multi_agent.py
```

Read:

```
interviewer_agent.py
```

Explore the existing Interviewer + Evaluator orchestration and extend it if desired.

This milestone is **optional and ungraded**.

---

8. Helper Files

The project includes several helper files that support the implementation.

`.env`

Stores API keys and other environment variables required by the application.

For example:

```
GROQ_API_KEY
```

`.env.example`

Provides an example structure for the required environment variables.

`.gitignore`

Specifies files and folders that Git should ignore, including the Python virtual environment.

`requirements.txt`

Contains the exact Python packages required by the project:

- `openai`
- `python-dotenv`
- `gradio`

---

9. Successful Completion Criteria

The project is considered successfully completed when the following workflow works correctly.

Running:

```
python app.py
```

should open the InterviewIQ browser interface.

The candidate should be able to:

1. Go through every question in `interview_bank.py`.
2. Submit an answer to each question.
3. Receive per-answer evaluation and feedback.
4. See a live scorecard containing all questions answered so far.
5. Ask the coach a free-form meta-question at any point during the session.
6. Receive an answer based on the complete session history.
7. Generate a final aggregated report.
8. See the average relevance score.
9. See the weakest area identified from the lowest-scoring question.

Additionally:

```
python memory_check.py
```

must pass successfully.

The memory check is particularly important because it verifies that InterviewIQ is actually remembering and aggregating the entire session rather than simply using the most recent answer.

---

10. Overall InterviewIQ Flow

The core system can be summarized as:

Candidate Answer
↓
Evaluator Agent
↓
Select Relevant Tools
↓
Filler Word Detection + STAR Analysis + Relevance Scoring
↓
Structured Evaluation Results
↓
Short Encouraging Feedback
↓
Session Memory
↓
Live Scorecard + Mid-Session Questions + Final Report

For the optional multi-agent system:

Interviewer Agent
↓
Analyze Candidate Performance
↓
Select Next Question Category
↓
Candidate Answers
↓
Evaluator Agent
↓
Evaluate Answer & Update Session Memory
↓
Interviewer Agent Selects Next Category

↓
Repeat

This creates a continuous interview loop in which the Interviewer determines what should be asked next, while the Evaluator focuses on assessing the candidate's current answer.